INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES
2. STUDENTS WILL NOT BE ALLOWED TO ENTER THE CLASS AFTER 9.30AM AND 11.00AM
3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS
4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK
5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY
6. SLEEPING IN CLASS IS TOTALLY PROHIBITED
7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS
8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

- BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES

- STUDENTS WILL NOT BE ALLOWED TO ENTER THE CLASS AFTER 9.30AM AND 11.00AM

SOC 2040
SYSTEM PROGRAMMING

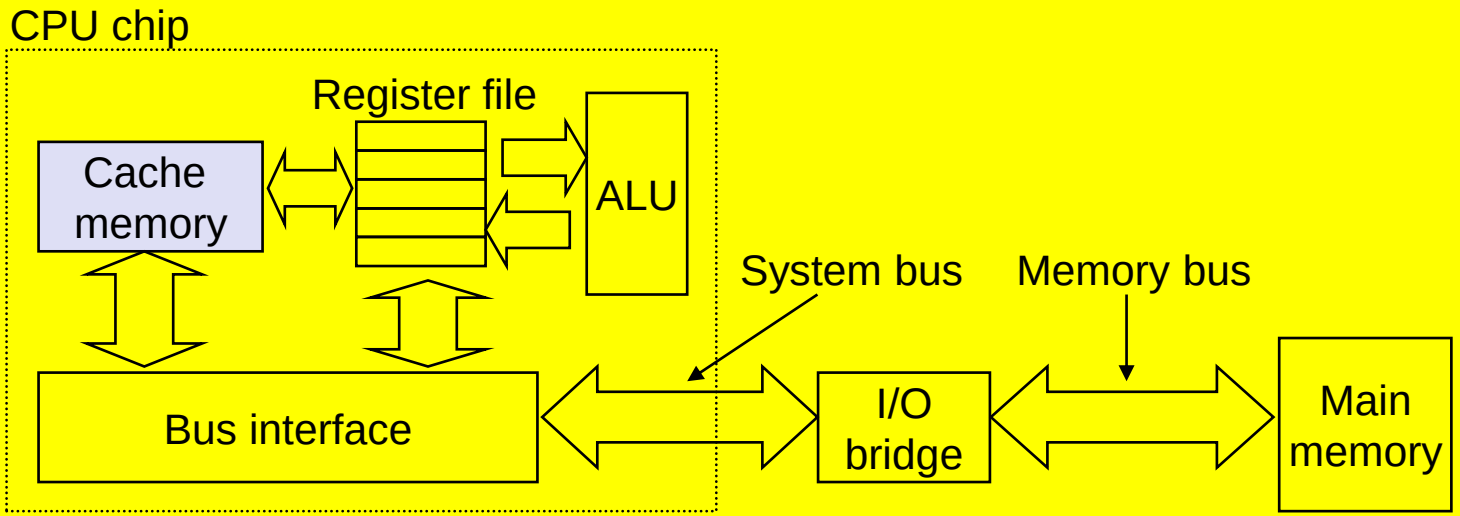
WEEK 11 LECTURE 1

MEMORY HIERARCHY PART 3

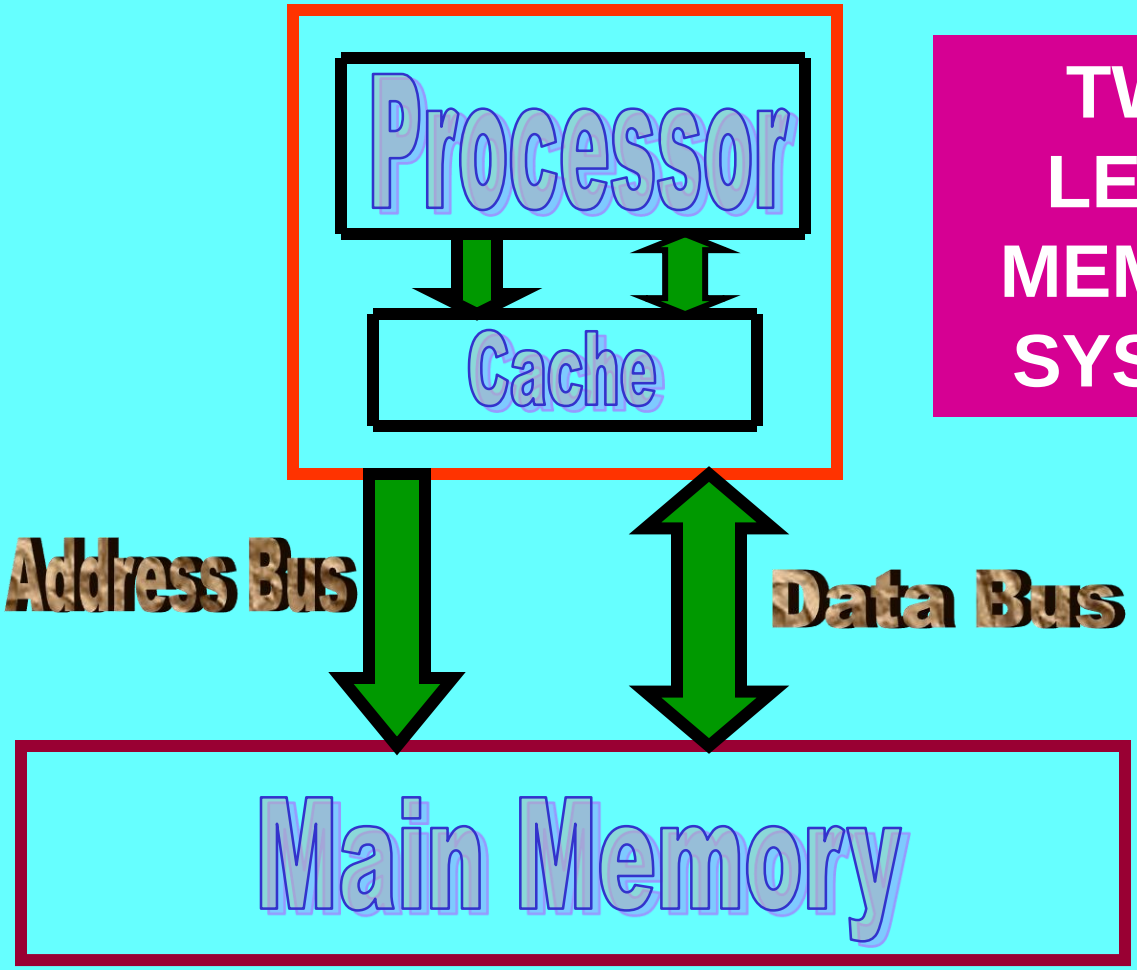
CACHE MEMORIES

Cache Memories

- Cache memories are small, fast SRAM-based memories managed automatically in hardware
  - Hold frequently accessed blocks of main memory
- CPU looks first for data in cache
- Typical system structure:



TWO-LEVEL MEMORY SYSTEM



Cache - a relatively small capacity but high speed memory inserted between the processor and the main memory

Purpose : The cache is introduced into the system

- **to decrease the effective memory access time**
- **to increase the operational speed of the system.**

Cache Memory

- **Program Characteristics**

- Program (code) is generally executed sequentially

- Virtually all programs repeat sections of code (i.e., contain loops) and repeatedly access the same or nearby data.

- These characteristics are embodied in the Principle of Locality

Cache Memory

- **Principle of Locality**
  - states that programs access a relatively small portion of their address space at any instant of time.
  - obeyed by most programs

- **There are two different types of locality:**
  - Temporal Locality
  - Spatial Locality

➢ Locality in programs arises from simple and natural program structures.

Principle of Locality

- Two kinds of Locality of Reference
  - Spatial Locality (Locality in Space)
  - Temporal locality (Locality in Time)

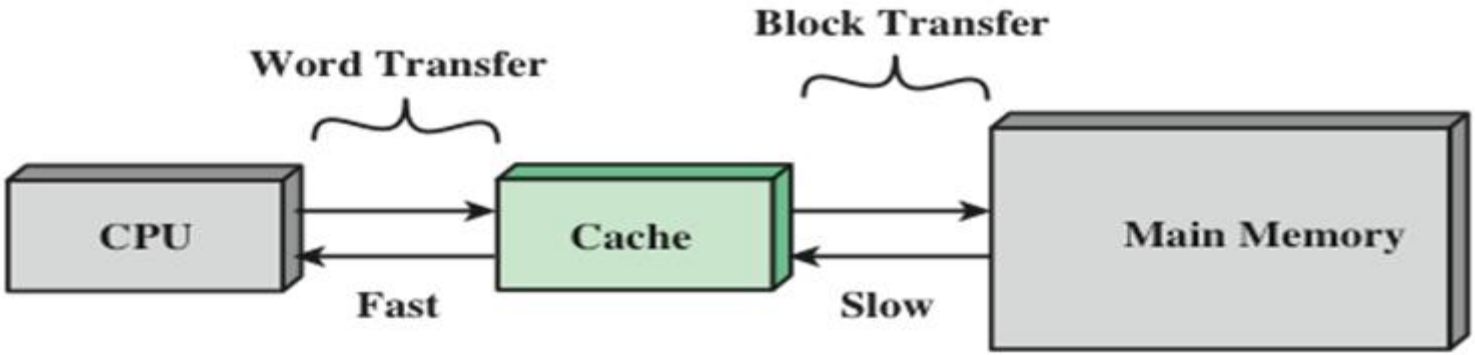
Spatial locality

- refers to the tendency of execution to involve a number of memory locations that are clustered.
- This reflects the tendency of a processor to access instructions sequentially.
- Spatial location also reflects the tendency of a program to access data locations sequentially, such as when processing a table of data.

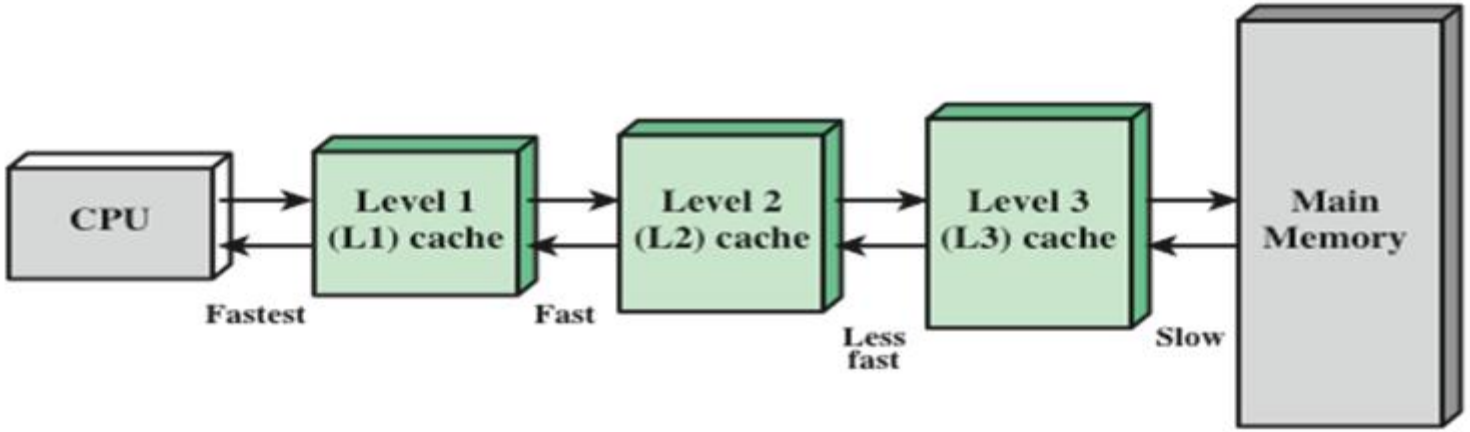
Temporal locality

- refers to the tendency for a processor to access memory locations that have been used recently.
- For example, when an iteration loop is executed, the processor executes the same set of instructions repeatedly.
- Traditionally, temporal locality is exploited by keeping recently used instruction and data values in cache memory and by exploiting a cache hierarchy.
- Spatial locality is generally exploited by using larger cache blocks and by incorporating prefetching mechanisms (fetching items of anticipated use) into the cache control logic.

Cache and Main Memory



(a) Single cache



(b) Three-level cache organization

Cache and Main Memory

CACHE MEMORY ORGANIZATIONS

Cache Memory Organization

- Cache Memory is organized as a collection of one or more sets
- Each Set is a collection of one or more blocks or lines
- Each Block or line is a collection of words
- Each Word is a collection of one or more bytes

Some Terms associated with a cache:

SET - COLLECTION OF LINES OR BLOCKS

BLOCK OR LINE - COLLECTION OF WORDS

WORD – COLLECTION OF BYTES

Mapping Function

- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines.
- Further, a means is needed for determining which main memory block currently occupies a cache line.
- The choice of the mapping function dictates how the cache is organized.
- Three techniques can be used:

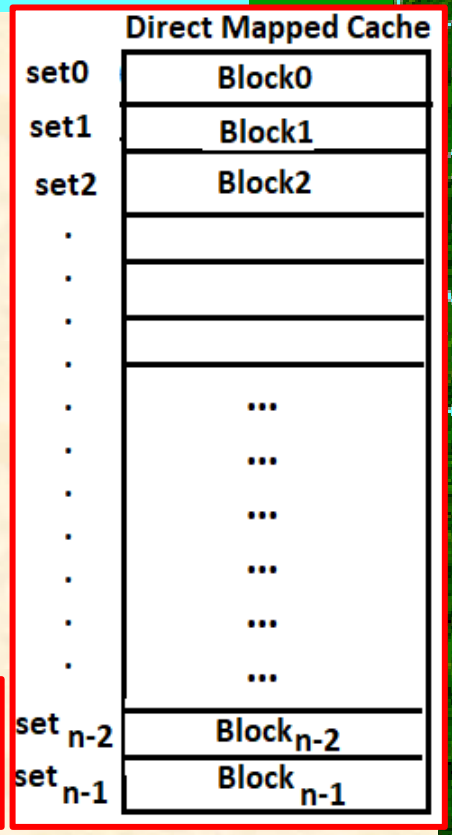
Direct, Fully Associative, and Set associative.

Cache Memory Organizations

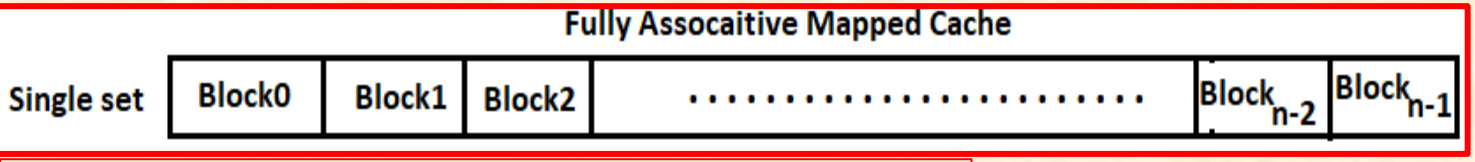
Direct Mapped Cache is a collection of more than one set where each set contains only one block or line.

A set contains only a single line or block or every block or line is a set.

If a cache contains n blocks then Direct mapped cache has n sets

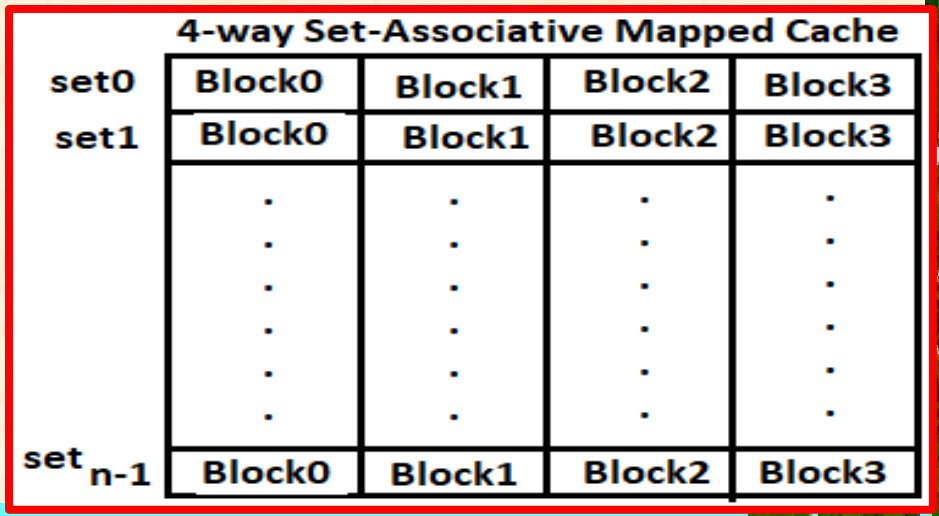


Fully Associative Mapped Cache has a single set containing all blocks. All blocks belong to a set. If a cache contains n blocks then Fully Associative mapped cache has a single set containing n blocks or lines



Set Associative Mapped Cache is a collection of more than one set where each set contains more than one block

If a cache contains n blocks then a 4-way Set Associative mapped cache has 4 blocks or lines in every set (cache contains n/4 sets)



DIRECT Mapped Cache

- In order to access the Direct Mapped Cache, the address from the processor is divided into three fields - **Tag, Index, & Byte Offset**

Memory Address from Processor

- The **Tag** field consists of the higher significant bits of the address, which is used to identify the block/line in the selected set in the cache
- The **Index** field is the next higher significant bits of the address which is used to address the set in the cache.
- The **Byte offset** field bits are used to access a byte within the selected Block or line.

Direct Mapped Cache Organization

Memory Address from Processor

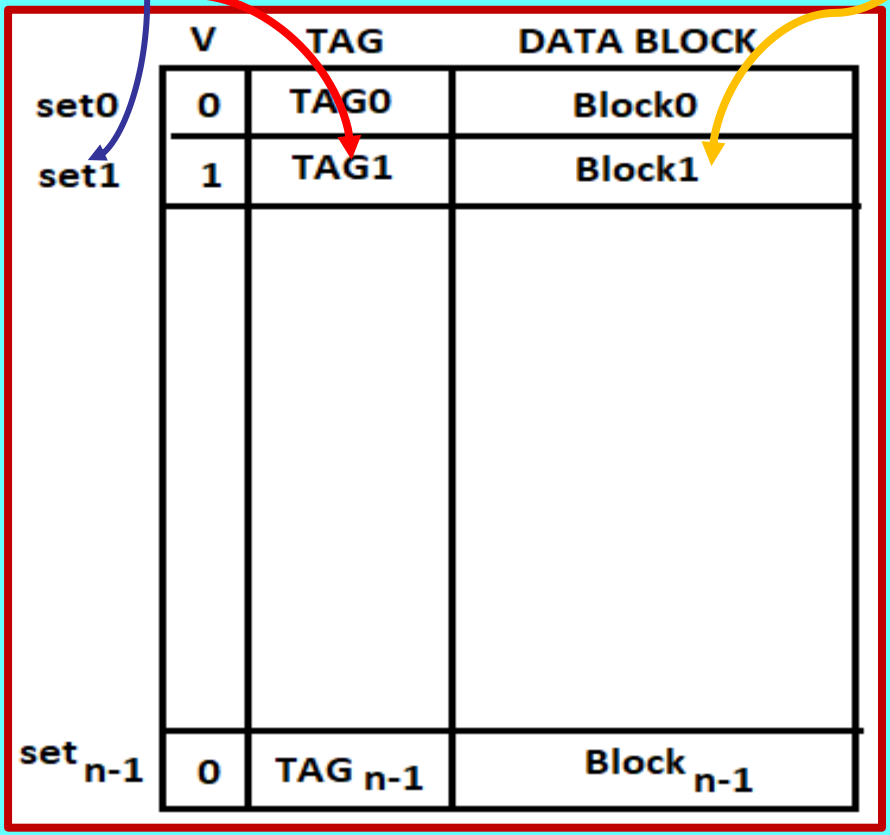
V is a 1-bit valid bit

This is used to indicate whether the block in a set is valid or not valid

V=1 indicates that set contains a valid block

V=0 indicates that set contains an invalid data block

Initially when the system is booted up, all these sets will have Valid bit=0



Direct Mapped Cache

TAG field is used in cache to indicate whether the Data block in that set is the required block or not

If the TAG in the tag field of a set matches with the TAG content in the TAG field of the Memory address generated by the processor then the data block in that set is the required block to be accessed

Direct Mapped Cache

- **Question**

- **A byte-addressable computer has a 16 MB main memory with a word size of 32 bits and a cache of 8 KB.**

  Determine the number of bits in each field of the memory address in the following organization :

  Direct mapping with a line or block size of one word

Direct Mapped Cache

Solution

Given : Size of Main Memory = 16 MB = 2^{24} bytes
Size of cache = 8 KB = 2^{13} bytes
Word size = 32 bits = 2^2 bytes
Line size = 1 word = 2^2 bytes

# of bits in Main memory address = 24 bits

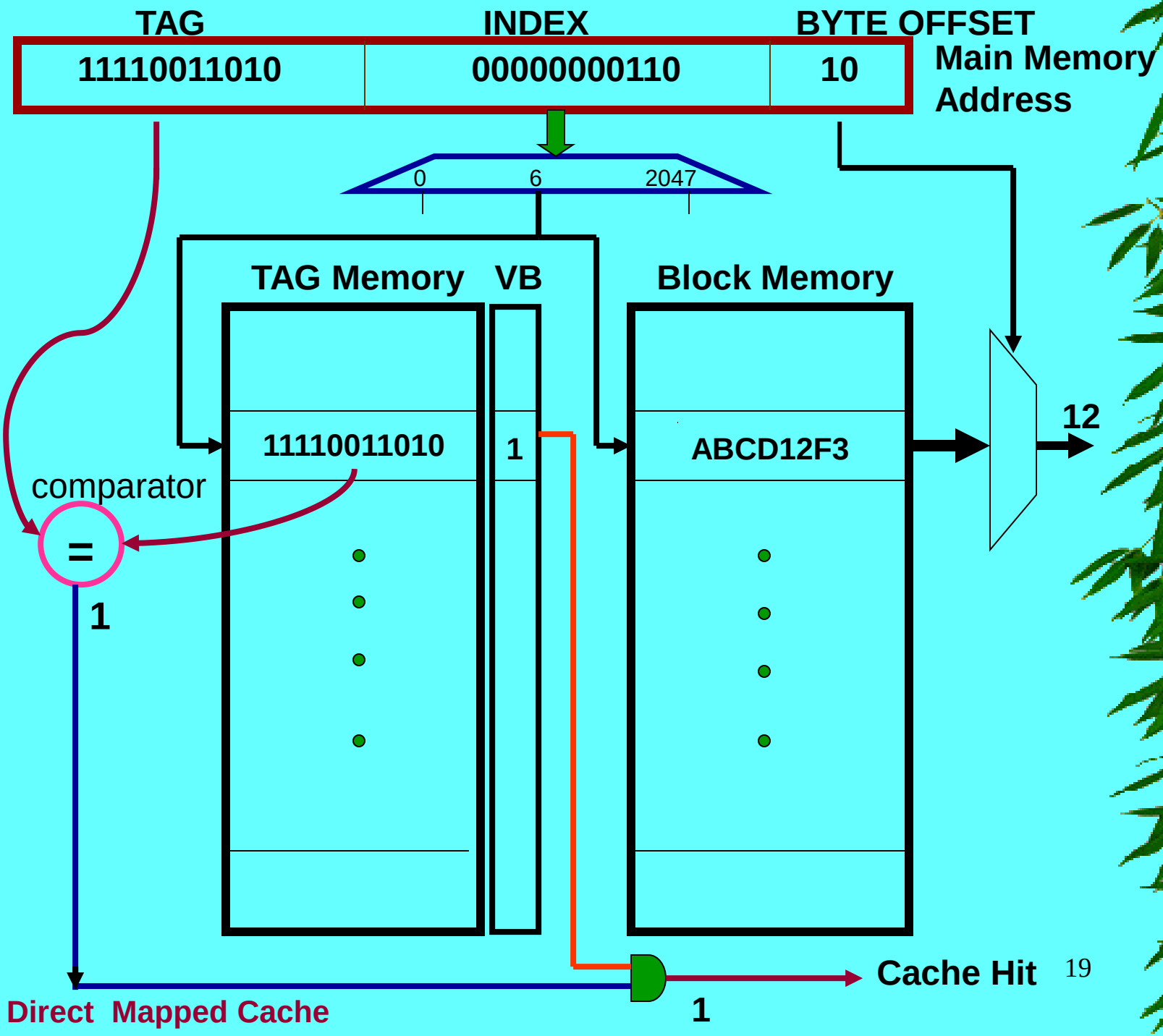
# of lines in cache = cache size / line size
= 2^{13} / 2^2 = 2^{11} lines

Hence, # of INDEX field bits = 11 bits

Line size = 1 word = 2^2 bytes

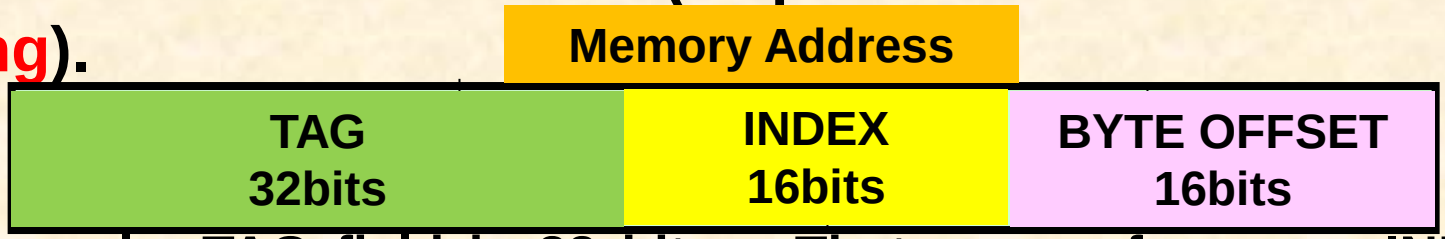
Hence, # of BYTE OFFSET field bits = **2** bits

# of TAG field bits = 24 – (11+2)
= **11** bits


TAG
INDEX
BYTE OFFSET
11110011010
00000000110
10
Main Memory Address
0
6
2047
TAG Memory
VB
Block Memory
11110011010
1
ABCD12F3
12
comparator
=
1
1
Cache Hit
Direct Mapped Cache

Direct Mapped Cache

- The corresponding lines with the same index in the main memory will map into the same line in the cache
- Thus, if a program happens to reference words repeatedly from two different blocks that map into the same line, then the blocks will be continually swapped in the cache, and the hit ratio will be low (a phenomenon known as thrashing).



- In this example, TAG field is 32 bits – That means for every INDEX value, there will be 2^{32} TAG combinations, or i.e 2^{32} combinations of memory addresses with the same INDEX, for example INDEX=0 will all map to the same line or block in set0. Hence there will large number misses and replacements
- Hence only lines with different indices can be in the cache at the same time.
- A Replacement algorithm is not necessary, since there is only one allowable location for each incoming line.

Direct Mapped Cache

Question :

Consider a 32-bit microprocessor that has an on-chip 4-KB direct mapped cache. Assume that the cache has a line (block) size of 32-bits (4 bytes or 2 words).

a)Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss.

b)Where in the cache is the word from memory location ABCDE8F8 mapped?

Answer :

Memory Address length = 32-bits, Main memory size = 2^{32}

Direct Mapped Cache of size = 4KB = 2^{12} bytes

line or block size = 32 bits = 4 bytes = 2^2 bytes

Number of blocks or lines in cache = 2^{12} / 2^2 = 2^{10} = 1024 blocks (lines)

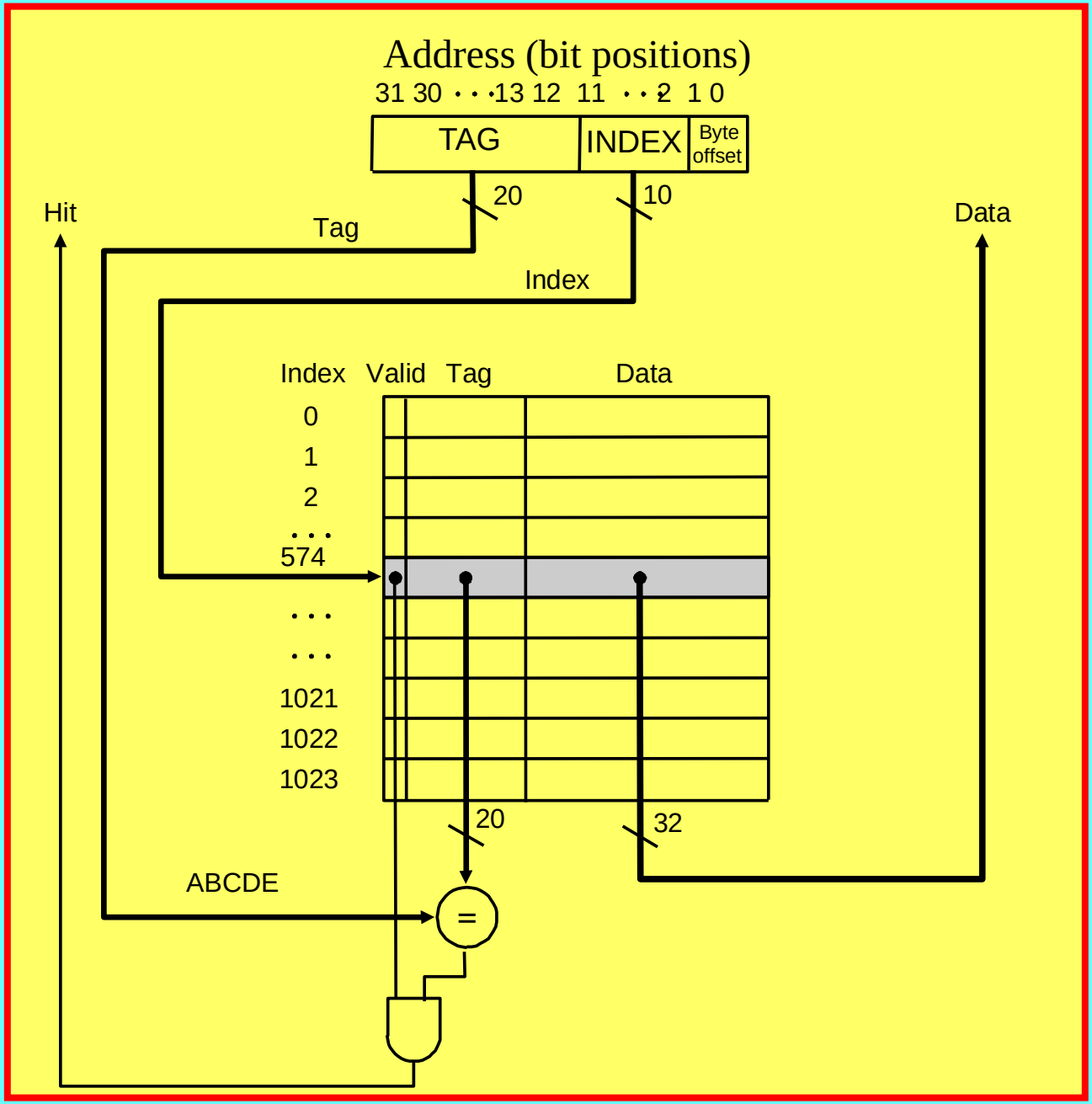
Number of bits to access a block or line in cache = INDEX bits = 10 bits

Byte OFFSET = number of bits to access a byte = 2 bits

TAG bits = Memory address length – (INDEX bits + Byte OFFSET bits)

= 32 – (10 + 2) = 20 bits

Direct Mapped Cache



Direct Mapped Cache

b) Where in the cache is the word from memory location ABCDE8F8 mapped?

Memory Address : ABCDE8F8

1010 1011 1100 1101 1110 | 1000 1111 10 | 00

BYTE OFFSET = 2 BITS
BYTE OFFSET = 00 - ACCESS BYTE 0

INDEX = 10 BITS
INDEX =1000111110 = 0x23E = BLOCK 574 (SET 574)

TAG BITS = 20 BITS
TAG = 10101011110011011110= ABCDE

Direct Mapped Cache

Question :

Consider a 32-bit microprocessor that has an on-chip 64-KB direct mapped cache. Assume that the cache has a line (block) size of 4 words where each word is 32-bits (4 bytes).

a) Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss.

b) Where in the cache is the word from memory location **9C8D0068** mapped?

Answer :

Memory Address length = 32-bits, Main memory size = 2^{32}

Direct Mapped Cache of size = 64KB = 2^{16} bytes

Word size = 32 bits = 4 bytes, line or block size = 4 words = 4 x 2^2 = 2^4 bytes

Number of blocks or lines in cache = 2^{16} / 2^4 = 2^{12} = 4096 blocks (lines)

Number of bits to access a block or line in cache = INDEX bits = 12 bits

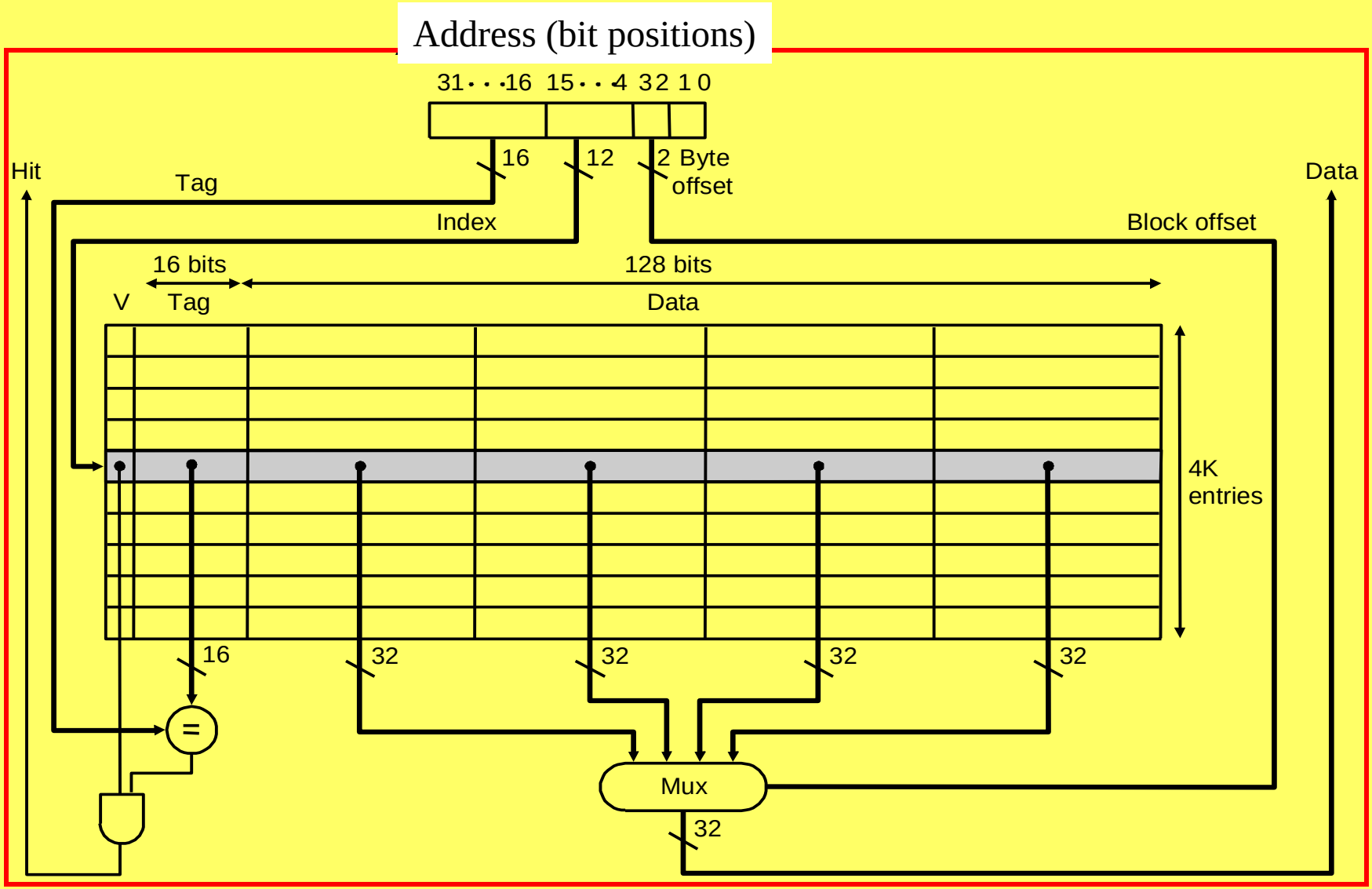
Byte OFFSET = number of bits to access a byte = 4 bits

TAG bits = Memory address length – (INDEX bits + Byte OFFSET bits)

= 32 – (12 + 4) = 16 bits

Direct Mapped Cache

64KB Cache with block or line of 4 words with word size = 32bits(4 bytes)



Direct Mapped Cache

b) Where in the cache is the word from memory location **9C8D0068** mapped?

Memory Address : **9C8D0068**

1001 1100 1000 1101	0000 0000 0110	1000

BYTE OFFSET = 4 BITS
BYTE OFFSET = 1000 - ACCESS BYTE 0 in word 2

INDEX = 12 BITS
INDEX = 000000000110 = 0x006 = BLOCK 6 (Set 6)

TAG BITS = 16 BITS
TAG = 1001110010001101= 9C8D

Direct Mapped Cache

Advantages

- Simple Hardware and Low cost
- High speed of operation
- No replacement algorithm necessary

Disadvantages

- Its main disadvantage is that there is a fixed cache location for any given memory block.
- Performance drops significantly if accesses are made to locations with the same index.
- Hit ratio lower than the associative mapping methods.
- Poor utilization of cache

Full Associative Mapped Cache

Fully Associative Mapping Approach

- A simple way of relating cached data to the main memory address is by **storing both the memory address and the corresponding data together in the cache**
- A fully associative cache requires the cache to be composed of

  - **Associative memory (Content Addressable Memory, CAM)** holding both the memory address and the data for each cached line. **Each cache block or line requires a separate comparator.**
- The incoming memory address is simultaneously compared with all stored addresses using the internal logic of the associative memory
- If a match is found, the corresponding data is read out.

Fully Associative Mapped Cache

- In order to access the Fully Associative Mapped Cache, the address from the processor is divided into two fields - **Tag and Byte Offset**

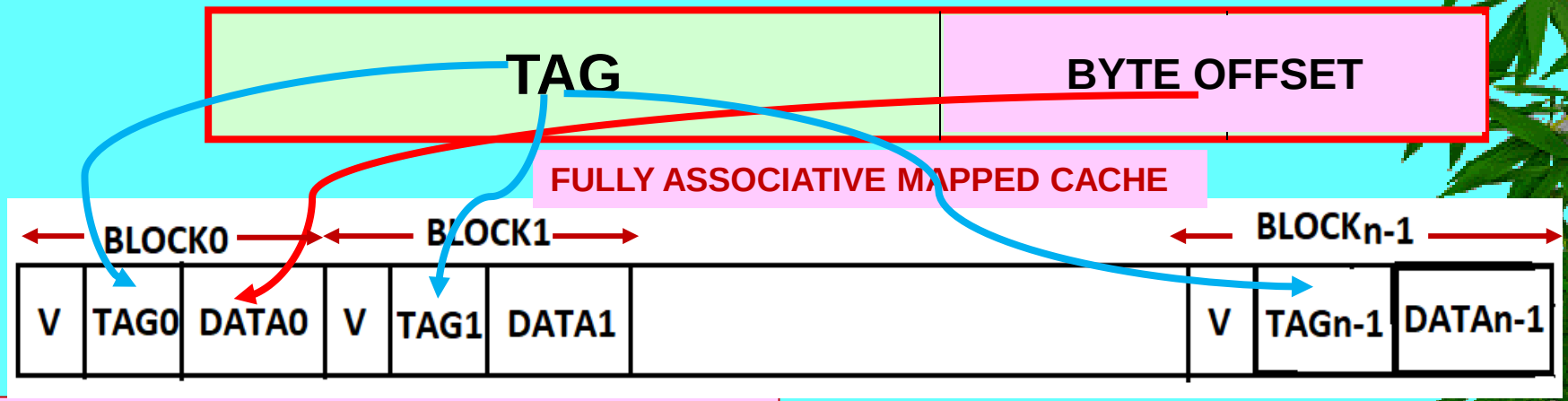
Memory Address from Processor

- The **Tag** field consists of the higher significant bits of the address, which is used to identify the line or block in the cache

- The **Byte** field bits are used to access a byte within the selected block or line.

Fully Associative Mapped Cache Organization

Memory Address from Processor



V is a 1-bit valid bit
Every block has a valid V bit

This is used to indicate whether the data block in a block is valid or not valid

V=1 indicates that block contains a valid block

V=0 indicates that block contains an invalid data block

Initially when the system is booted up, all these blocks will have Valid bit=0

Fully Associative Mapped Cache has a single set containing n blocks/lines

TAG field is used in cache to indicate whether the Data block in the block is the required block or not

If the TAG in the tag field of the block matches with the TAG content in the TAG field of the Memory address generated by the processor then the data block in that block is the required block to be accessed

FULLY ASSOCIATIVE MAPPING

- Fully Associative mapping overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of the cache.
- In this case, the cache control logic interprets a memory address simply as a Tag and a Byte offset field.
- The Tag field uniquely identifies a block of main memory.
- To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's tag for a match.

Full Associative Mapped Cache

- **Question**

- **A byte-addressable computer has a 16 MB main memory with a word size of 32 bits and a cache of 8 KB.**

  Determine the number of bits in each field of the memory address in the following organization :

  Fully Associative mapping with a line or block size of eight words

Full Associative Mapped Cache

- **Solution**

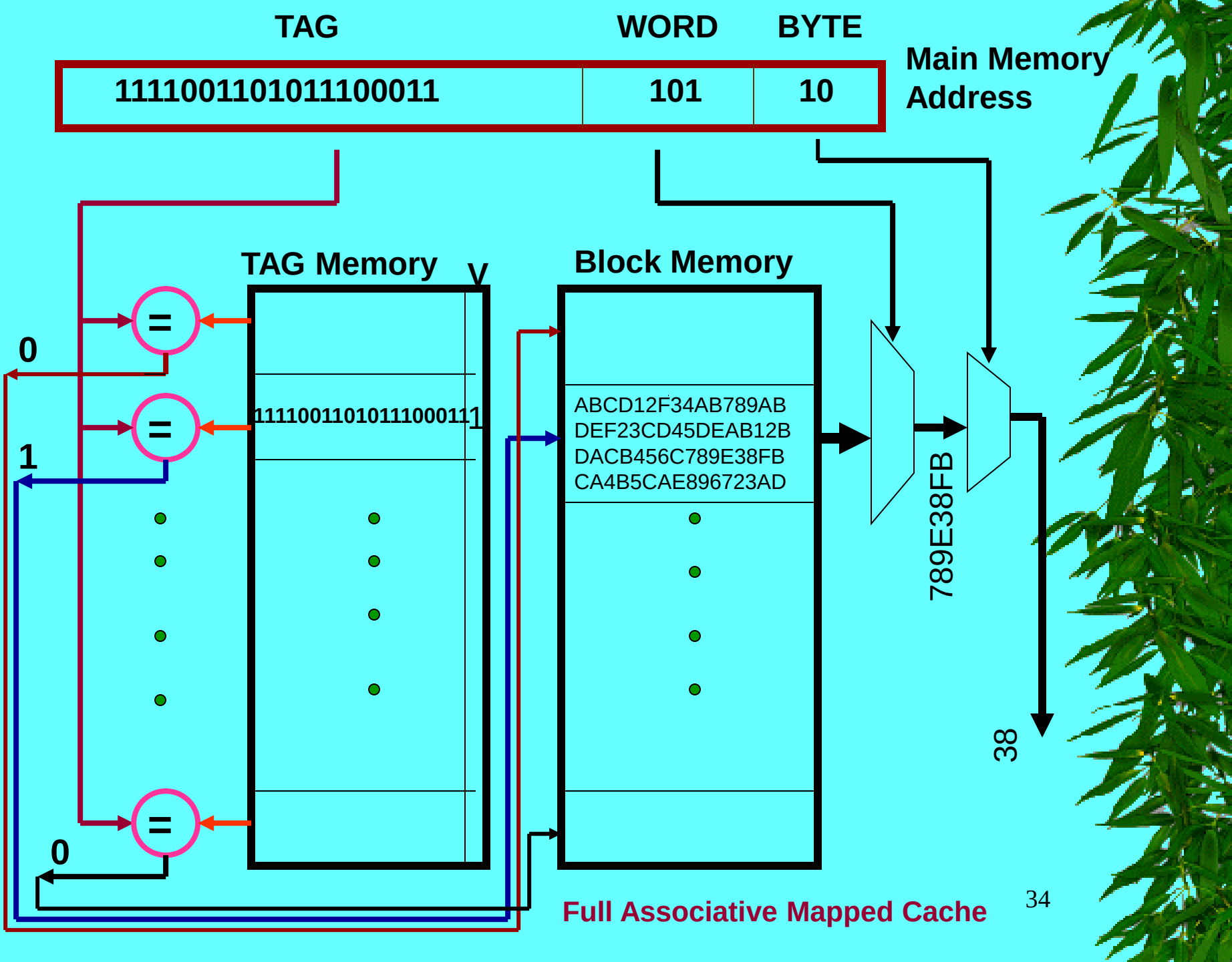
- **Given :** Size of Main Memory = 16 MB = 2^{24} bytes
  Size of cache = 8 KB = 2^{13} bytes
  Word size = 32 bits = 2^2 bytes
  Line or block size = 8 words = 2^3 words = 2^5 bytes
  # of blocks in cache = cache size / line size
  = 2^{11} / 2^5 = 2^6 lines

# of bits in Main memory address = 24 bits

# of bytes in a line or block = 2^5 bytes

Hence, # of BYTE OFFSET field bits = **5** bits

# of TAG field bits = 24 – 5= **19** bits



Full Associative Mapped Cache

Fully Associative Mapped Cache

Question :

Consider a 32-bit microprocessor that has an on-chip 4-KB Fully Associative mapped cache. Assume that the cache has a line (block) size of 32-bits (4 bytes or 2 words).

a) Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss.

b) Where in the cache is the word from memory location ABCDE8F8 mapped?

Answer :

Memory Address length = 32-bits, Main memory size = 2^{32}

Fully Associative Mapped Cache of size = 4KB = 2^{12} bytes

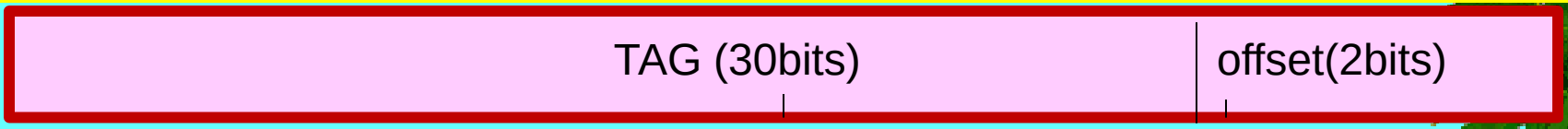
line or block size = 32 bits = 4 bytes = 2^2 bytes

Number of blocks or lines in cache = 2^{12} / 2^2 = 2^{10} = 1024 blocks (lines)

Byte OFFSET = number of bits to access a byte = 2 bits

TAG bits = Memory address length – Byte OFFSET bits

= 32 – 2 = 30 bits



Fully Associative Mapped Cache

b) Where in the cache is the word from memory location ABCDE8F8 mapped?

Memory Address : ABCDE8F8

1010 1011 1100 1101 1110 1000 1111 1000

BYTE OFFSET = 2 BITS
BYTE OFFSET = 00 - ACCESS BYTE 0

TAG BITS = 30 BITS
TAG = 10101011110011011110100011110= 2AF37A3E

Fully Associative Mapped Cache

Question :

Consider a 32-bit microprocessor that has an on-chip 64-KB Fully Associative mapped cache. Assume that the cache has a line (block) size of 4 words where each word is 32-bits (4 bytes).

a) Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss.

b) Where in the cache is the word from memory location **9C8D0068** mapped?

Answer :

Memory Address length = 32-bits, Main memory size = 2^{32}

Fully Associative Mapped Cache of size = 64KB = 2^{16} bytes

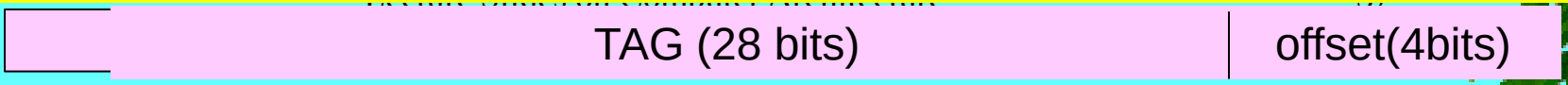
Word size = 32 bits = 4 bytes, line or block size = 4 words = 4 x 2^2 = 2^4 bytes

Number of blocks or lines in cache = 2^{16} / 2^4 = 2^{12} = 4096 blocks (lines)

Byte OFFSET = number of bits to access a byte = 4 bits

TAG bits = Memory address length – Byte OFFSET bits

= 32 – 4 = 28 bits



Fully Associative Mapped Cache

b) Where in the cache is the word from memory location **9C8D0068** mapped?

Memory Address : **9C8D0068**

1001 1100 1000 1101 0000 0000 0110 1000

BYTE OFFSET = 4 BITS
BYTE OFFSET = 1000 - ACCESS BYTE 0 in word 2

TAG BITS = 28 BITS
TAG = 10011100100011010000000000110= 9C8D006

Full Associative Mapped Cache

Advantages

- Provides greatest flexibility of holding combinations of lines in the cache since block can be placed anywhere in the cache
- Minimum conflict for a given sized cache
- Higher hit ratio
- High Utilization of cache

Disadvantages

- Complex approach - complex circuitry required to examine the tags of all cache lines in parallel.
- Most Expensive due to the cost of the associative memory
- Requires a Replacement algorithm to select a line to remove upon a miss
- Algorithm must be implemented in hardware to maintain a high speed of operation

Set-Associative Mapped Cache

Set-Associative Mapping Approach

- Set-associative mapping allows a limited number of lines, with the same index and different tags, in the cache
- It can be considered as a compromise between a fully associative cache and a direct mapped cache
- The cache is divided into “sets” of lines
- A four-way set associative cache would have four lines in each set.
- The number of lines in a set is known as the associativity or setsize
- Each line in each set has a stored tag which, together with the set (index number), completes the identification of the line.

Set-Associative Mapped Cache

- In order to access the Set Associative Mapped Cache, the address from the processor is divided into three fields - **Tag, Set or Index, Byte Offset**

Memory Address from Processor

- The **Tag** field consists of the higher significant bits of the address, which is used to identify the line in the selected set in the cache

- The **Set** or **Index** field is the next higher significant bits of the address which is used to address the set in the cache.

- The **Byte offset** field bits are used to access a byte within the selected block or line.

4-way Set Associative Mapped Cache Organization

Memory Address from Processor

4-way Set Associative Mapped Cache

	V	TAG	DATA BLOCK 0	V	TAG	DATA BLOCK 1	V	TAG	DATA BLOCK 2	V	TAG	DATA BLOCK 3
set0	0	TAG0	Block0	0	TAG0	Block1	0	TAG2	Block2	0	TAG3	Block3
set1	1	TAG0	Block0	0	TAG1	Block1	0	TAG2	Block2	0	TAG3	Block3
set_{n-1}	1	TAG0	Block0	0	TAG	Block1	0	TAG2	Block2	0	TAG3	Block3

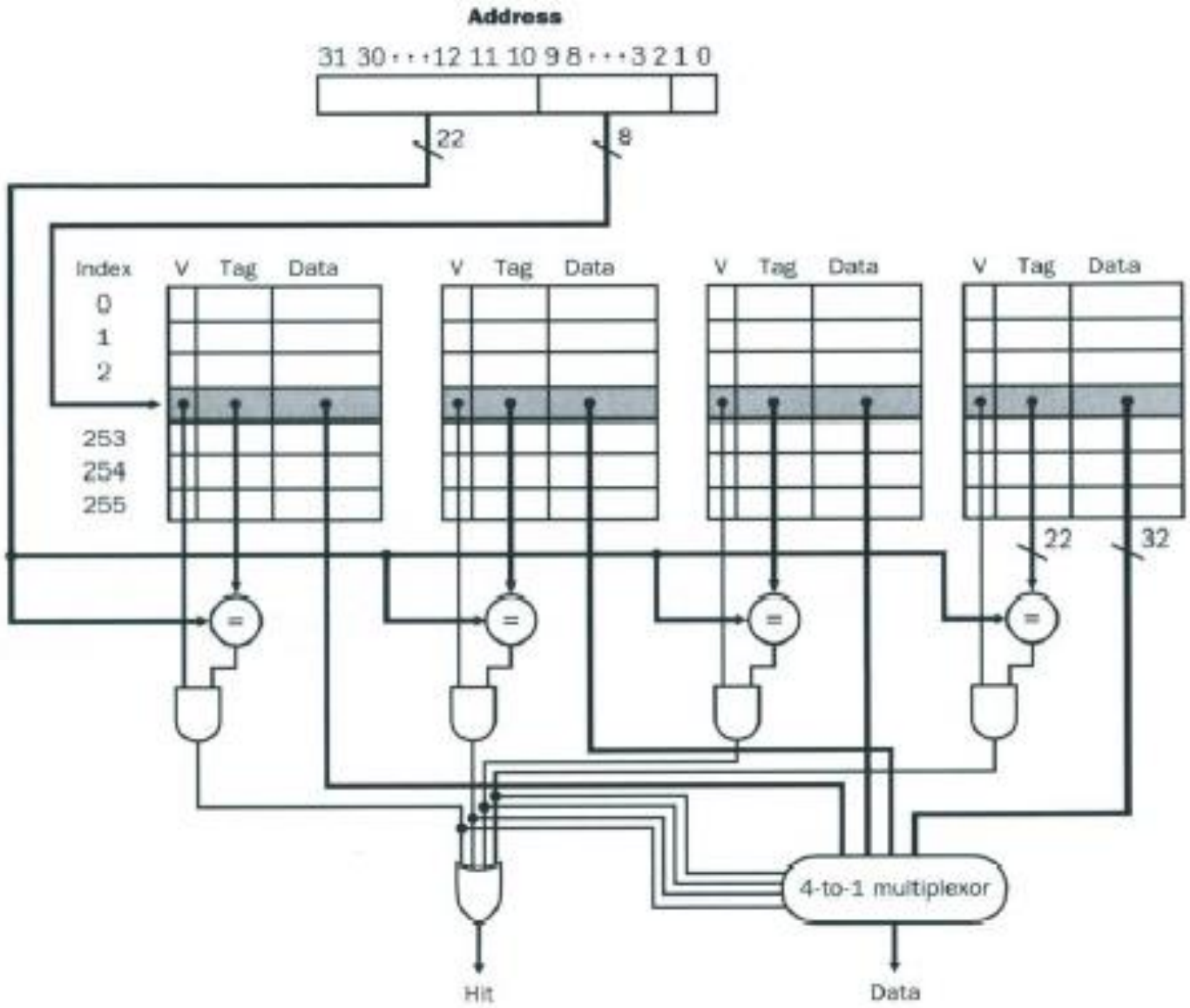
V is a 1-bit valid bit
This is used to indicate whether the block in a set is valid or not valid
V=1 indicates that block in a set contains a valid block
V=0 indicates that block in a set contains an invalid data block
Initially when the system is booted up, all the blocks in all sets will have Valid bit=0

TAG field is used in cache to indicate whether the Data block in that block in the set is the required block or not
If the TAG in the tag field of a block in a set matches with the TAG content in the TAG field of the Memory address generated by the processor then the data block in that block in the selected set is the required block to be accessed

Set-Associative Mapped Cache

- First, the Set (or Index field bits) of the address from the processor is used to access the set.
- Then, comparators are used to compare all tags of the selected set with the incoming tag.
- If a match is found, the corresponding location is accessed, otherwise, an access to the main memory is made.
- The number of comparators required in the set-associative cache is given by the number of lines in a set.
- The replacement algorithm for set-associative mapping need only consider the lines in one set, as the choice of set is predetermined by the Set field bits in the address.

Set-Associative Mapped Cache



Set-Associative Mapped Cache

- **Question**

- **A byte-addressable computer has a 16 MB main memory with a word size of 32 bits and a cache of 8 KB.**

  Determine the number of bits in each field of the memory address in the following organization :

  Set-associative mapping with a set size of two lines or blocks and block size of one word

Set-Associative Mapped Cache

- **Solution**

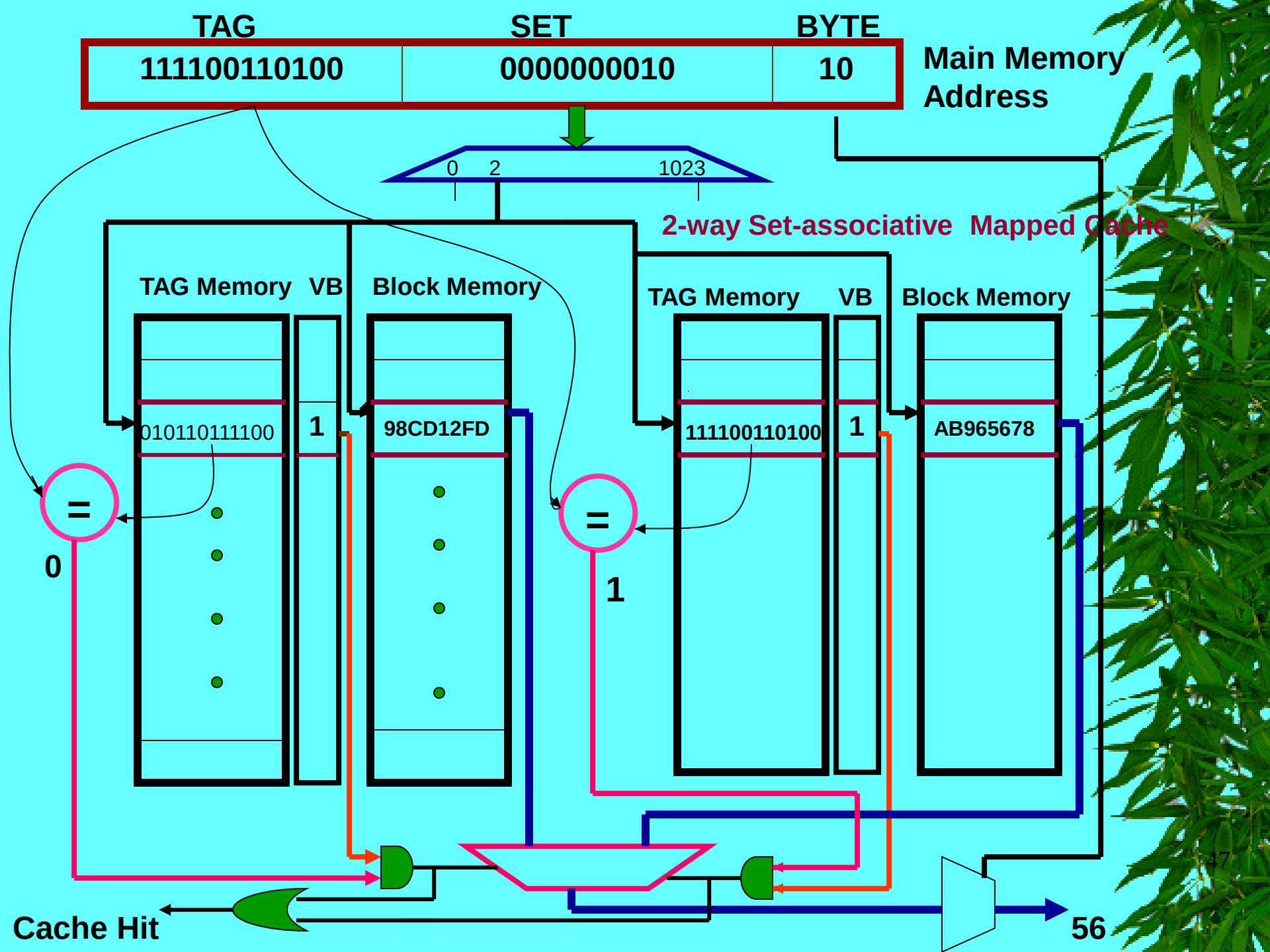
Given : Size of Main Memory = 16 MB = 2^{24} bytes
Size of cache = 8 KB = 2^{13} bytes
Word size = 32 bits = 2^2 bytes
Line size = 1 word = 2^2 bytes

Set size = 2 lines

# of bits in Main memory address = 24 bits
# of lines in cache = cache size / line size
= 2^{13} / 2^2 = 2^{11} lines

# of sets in cache = # of lines in cache / set size
= 2^{11} / 2 = 2^{10} sets
Hence, # of SET field bits = **10** bits

Line size = 1 word = 2^2 bytes
Hence, # of BYTE OFFSET field bits = **2** bits
# of TAG field bits = 24 – (10+2)
= **12** bits


TAG
SET
BYTE
111100110100
0000000010
10
Main Memory Address
0
2
1023
2-way Set-associative Mapped Cache
TAG Memory
VB
Block Memory
TAG Memory
VB
Block Memory
010110111100
1
98CD12FD
111100110100
1
AB965678
=
=
0
1
Cache Hit
56

Set Associative Mapping

- Set-associative mapping is a Compromise that exhibits the strengths of both the direct and fully associative approaches while reducing their disadvantages
- Cache consists of a number of sets
- Each set contains a number of lines
- A given block maps to any line in a given set
- e.g. 2 lines per set
  - 2 way associative mapping
  - A given block can be in one of 2 lines in only one set

4-way Set Associative Mapped Cache

Question :

Consider a 32-bit microprocessor that has an on-chip 4-KB 4-way Set Associative mapped cache. Assume that the cache has a line (block) size of 32-bits (4 bytes or 2 words).

Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss.

Where in the cache is the word from memory location ABCDE8F8 mapped?

Answer :

Memory Address length = 32-bits, Main memory size = 2^{32}

4-way Set Associative Mapped Cache of size = 4KB = 2^{12} bytes

line or block size = 32 bits = 4 bytes = 2^2 bytes

Number of blocks or lines in cache = 2^{12} / 2^2 = 2^{10} = 1024 blocks (lines)

Number of blocks per set = 4,

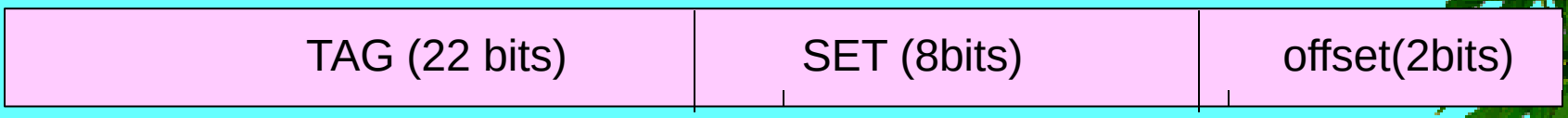
Number of sets in cache = 2^{10} / 2^2 = 2^8

Byte OFFSET = number of bits to access a byte = 2 bits

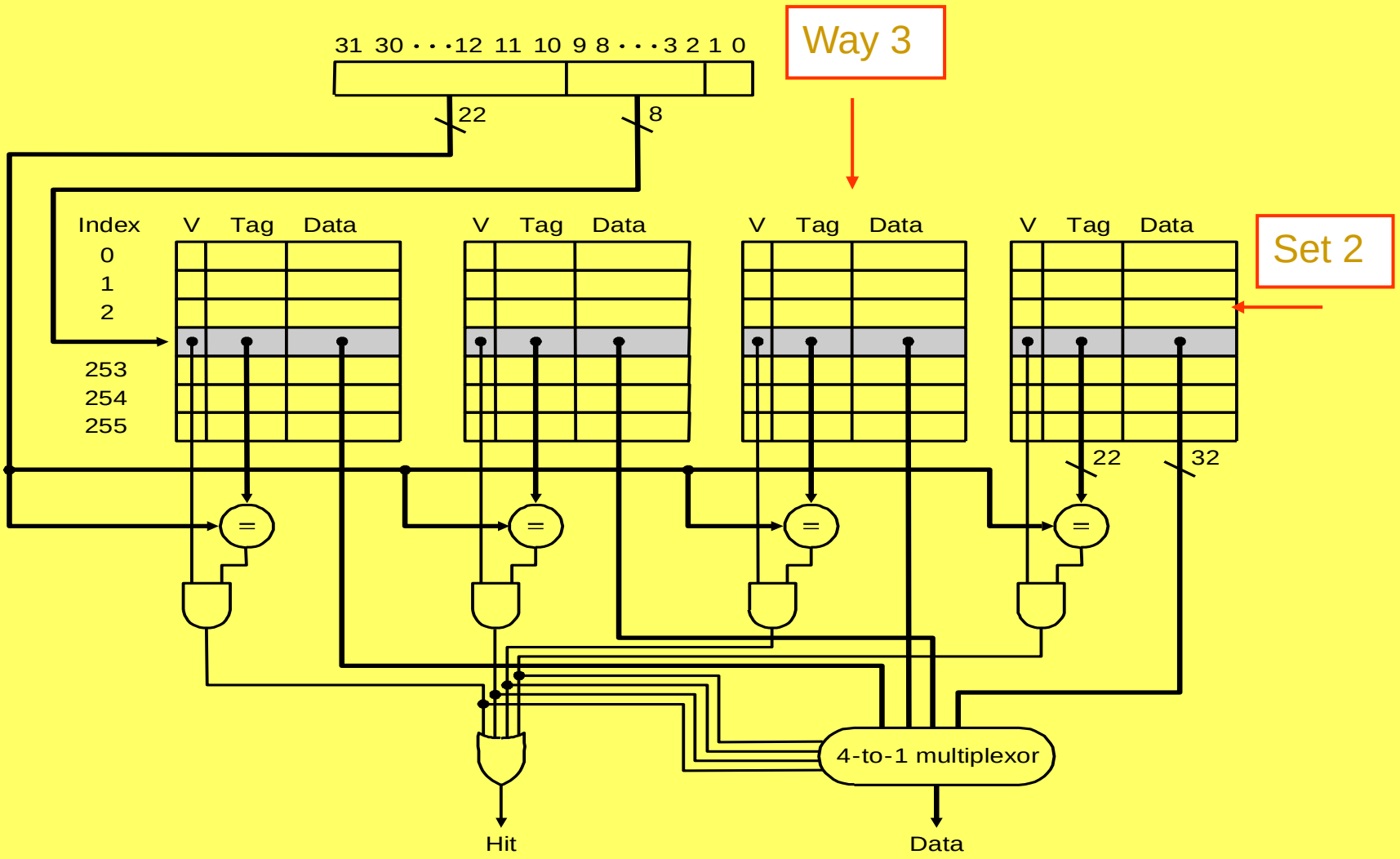
Number of SET field bits = 8

TAG bits = Memory address length – (SET bits + Byte OFFSET bits)

= 32 – (8+ 2) = 22 bits



4-Way Set-Associative Cache



4-way Set Associative Mapped Cache

b) Where in the cache is the word from memory location ABCDE8F8 mapped?

Memory Address : ABCDE8F8

1010 1011 1100 1101 1110 1000 1111 1000

BYTE OFFSET = 2 BITS
BYTE OFFSET = 00 - ACCESS BYTE 0

SET BITS = 8
SET FIELD = 00111110 = 0x3E = SET 62

TAG BITS = 22 BITS
TAG = 1010101111001101111010= 2AF37A

Set-associative Mapped Cache

- **Advantages**

- **Moderate hardware cost as the number of comparators required is equal to number of lines in the set.**
- **Moderate speed of operation**
- **Moderate utilization of cache**
- **Replacement algorithm necessary but need to consider only the lines in one set.**
- **Hit ratio better than the direct mapping method**

Cache Write Operation

Cache Write Operation

- After a write operation to the cache, it is possible that the cache word and copy held in the main memory may be different.
- Cache Coherence
  - It is necessary to keep the cache and the main memory copy identical if input/output transfers operate on the main memory contents, or if multiple processors operate on the main memory, as in a shared memory multiple processor system.
  - Coherence between a cache word and its copy in the main memory should be maintained at all times, if at all possible.

Cache Write Policies

- Determine the degree of coherence that can be maintained between cache words and their counterparts in the main memory

- Two main Cases

  - Cache write policies upon a Cache Hit

  - Cache write policies upon a Cache Miss

Cache Write Policies upon a Cache Hit

- Basically two possible write policies upon a cache Hit

- Write-Through Policy

- Write-back Policy

Cache Write Policies upon a Cache Hit

Write-Through Policy

- **Every write operation to the cache is repeated to the main memory at the same time.**

  i.e., the cache location and the main memory location are updated simultaneously.

- **The additional write operation to the main memory will take much longer than to the cache and will dominate the access time for write operations**
- **It maintains coherence between the cache blocks and their counterparts in the main memory at the expense of the extra time needed to write to the main memory**
- **This leads to an increase in the average access time**

Cache Write Policies upon a Cache Hit

❑ Write-back Mechanism

- **All writes are made only to the cache.**
- **Write to the main memory is postponed until a replacement is needed.**

  i.e., the write operation to the main memory is only done at line or block replacement time.
- **At this time, the block displaced by the incoming block might be written back to the main memory.**

Cache Write Policies upon a Cache Hit

Write-back Policy

- Every cache block is assigned a bit, called the dirty bit, to indicate that atleast one write operation has been made to the block while residing in the cache.
- At replacement time, the dirty bit is checked, if it is set, then the block is written back to the main memory, otherwise it is simply overwritten by the incoming block.
- This policy eliminates the increase in the average access time, however, coherence is only guaranteed at the time of replacement.

Cache Write Policies upon a cache miss

- Two main Schemes

- Allocate on write (Fetch on write)

- Non-allocate on write (No fetch on write)

Cache Write Policies upon a cache miss

- **Allocate on write (Fetch on write)**

  - **The requested word/line is brought from the main memory into the cache for a write operation on a cache miss and then updated**

- **Non-Allocate on write (No Fetch on write)**

  - **The word/line is written back to the main memory (i.e., updated) and not brought to the cache for a write operation on a cache miss**

Cache Write Policies

- **Write-through mechanism uses**

  no fetch on write (Non-allocate on write) policy

- **Write-back mechanism uses**

  fetch on write (Allocate on write) policy

Write Through and Write Back

- **Write through**
  - Simplest technique
  - All write operations are made to main memory as well as to the cache
  - Any other processor–cache module can monitor traffic to main memory to maintain consistency within its own cache.
  - The main disadvantage of this technique is that it generates substantial memory traffic and may create a bottleneck

- **Write back**
  - Minimizes memory writes
  - Updates are made only in the cache.
  - When an update occurs, a dirty bit, or use bit, associated with the line is set.
  - Then, when a block is replaced, it is written back to main memory if and only if the dirty bit is set.
  - Portions of main memory are invalid and hence accesses by I/O modules can be allowed only through the cache
  - This makes for complex circuitry and a potential bottleneck

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapters 6 & 7 of Text Book

- **Computer Systems : A Programmer's Perspective**

  - **Randal E. Bryant and David R. O'Hallaron**
    3rd Edition, Global Edition Prentice Hall 2016
    ISBN 10:1-292-10176-8

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

25 April 2024